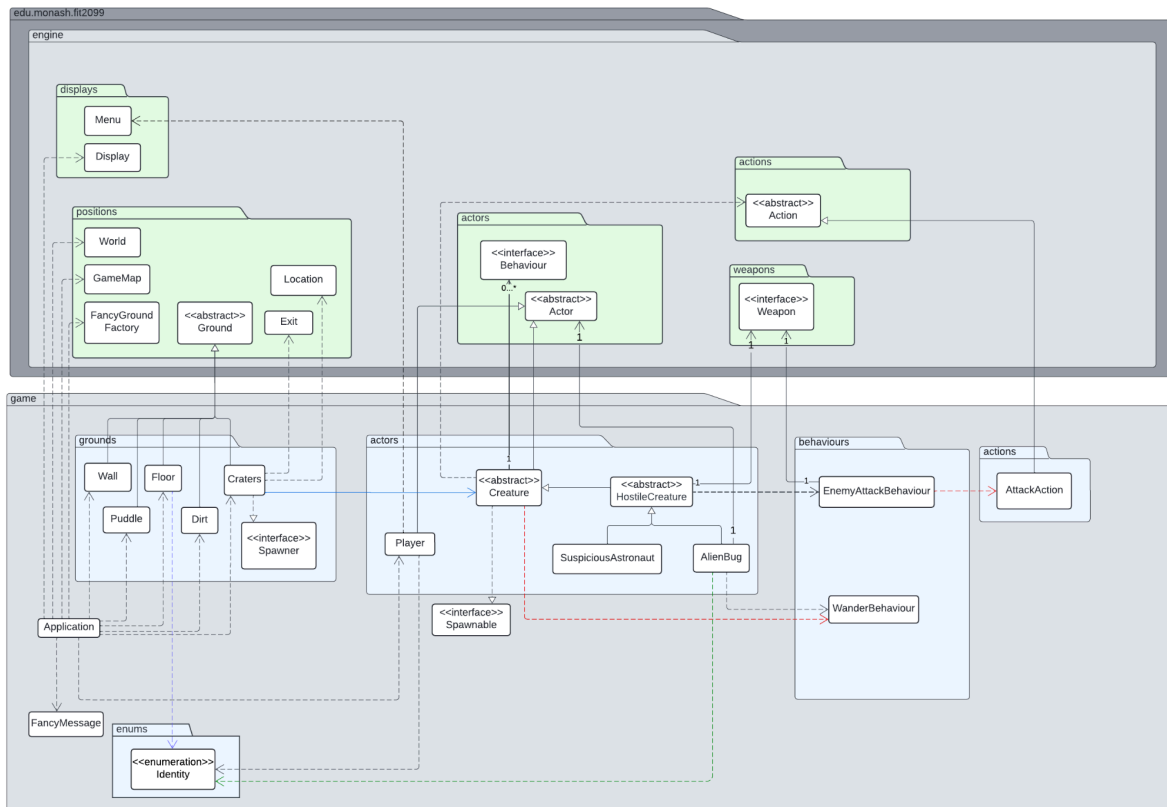


<https://docs.google.com/document/d/1DTMI4VE-4-Vvcl0tnlupAVaw1Uw9ZrQhQGV4nlxpkWs/edit?usp=sharing>

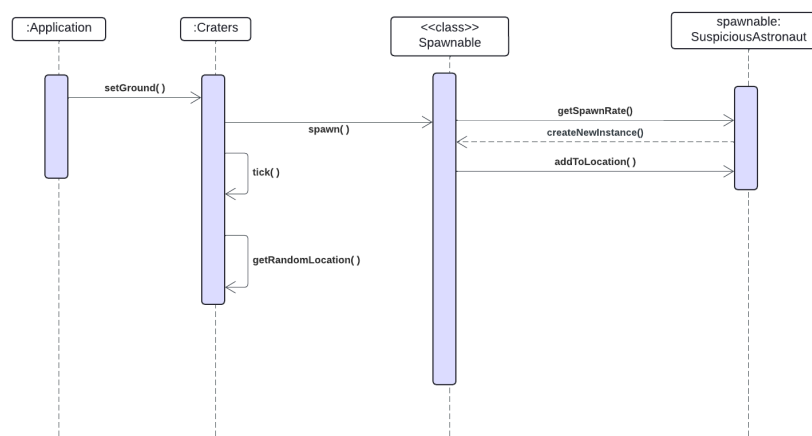
Requirement 1: The moon's (hostile) fauna II: The moon strikes back

UML Diagram:



This sequence diagram is specific to the scenario which the moon's craters spawn Suspicious Astronaut

Req1 seq



Design Goal:

The design goal for this requirement is to introduce additional creatures to the moon's craters, allowing them to spawn from craters, while adhering closely to relevant design principles and best practices.

Design Decision:

In implementing this requirement, the decision was made to be centred around the Crater, Creature and Hostile Creature class. As Alien Bug has no attack ability, it inherits the constructor and `createNewInstance()` method from the Creature class. While Suspicious Astronaut has attack ability, it inherits the constructor and `createNewInstance()` method from the Hostile creature class.

Alternative Design:

One alternative approach could involve a class structure where each creature type is managed independently without utilising a clear hierarchy of parent classes like Creature and Hostile Creature. However, inheritance simplifies the design by directly linking behaviours to specific creature types.

Analysis of Alternative Design:

The alternative design is not ideal because it violates various Design and SOLID principles:

1. Single Responsibility Principle:

- Each creature needs to handle its spawning, movement and other functionalities independently. This leads to classes that have multiple reasons to change.

2. Open Closed Principles:

- Adding a new creature could require changes to existing code rather than extending existing classes.

3. Don't Repeat Yourself:

- The functionality like `spawn()` is implemented separately for each creature, which leads to redundant code that performs similar tasks across different classes.

Final Design:

In our design, we closely adhere to OCP and LSP

1. Open Closed Principle:

- **Application:** The Creature and Hostile Creature class is open for extension but closed for modification.
- **Why:** New Creature types can be added by inheriting the Creature or Hostile Creature class without altering the existing class.
- **Pros/Cons:** This approach promotes code reusability and maintainability by easy addition of new creatures.

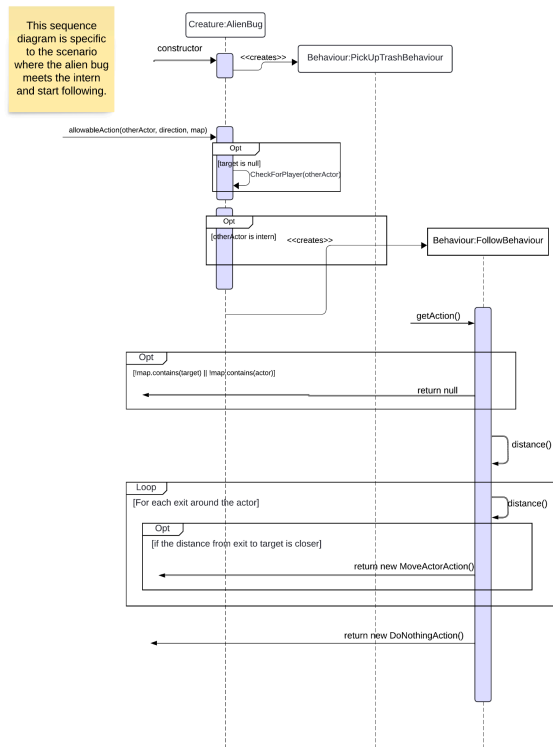
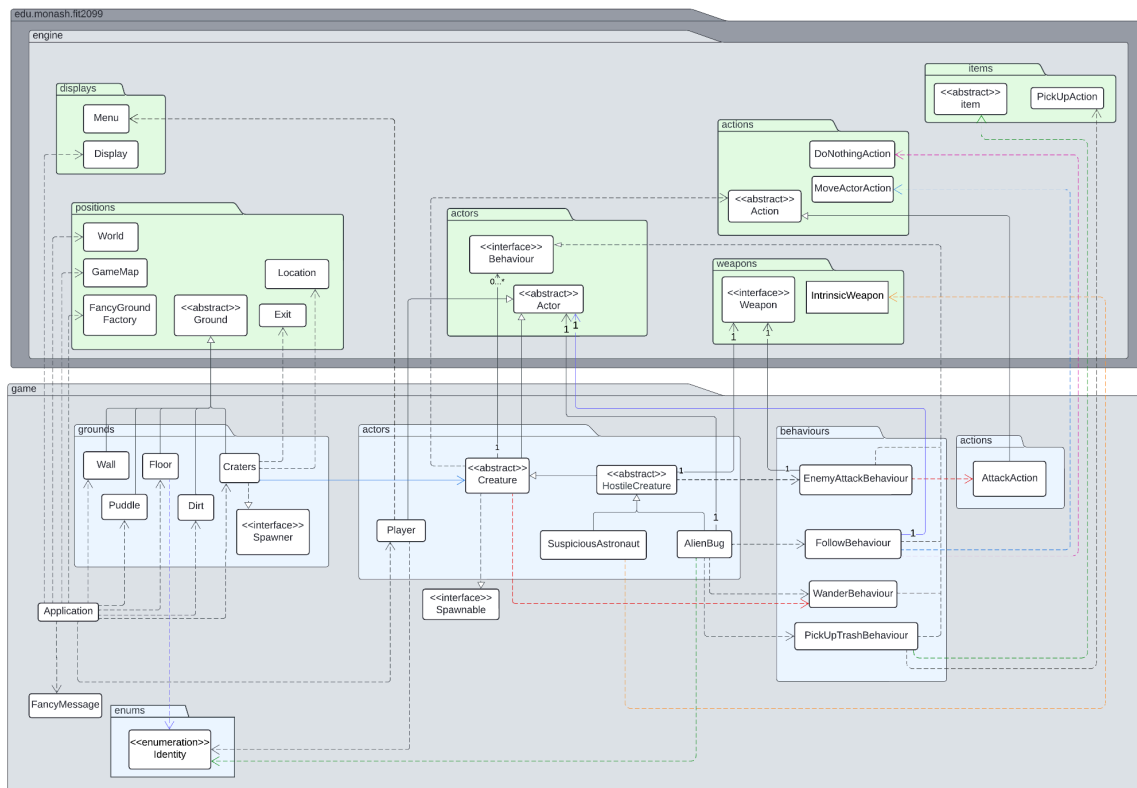
2. Liskov Substitution Principle:

- **Application:** Alien Bug and Suspicious Astronaut can be substituted for their parent class (Creature and Hostile Creature) without breaking the expected behaviour of its superclass.
- **Why:** It ensures that the Alien Bug and Suspicious Astronaut class maintain the expected behaviour of their parent class.
- **Pros/Cons:** Enhances code flexibility and extensibility

Conclusion:

Overall, our chosen design provides a robust framework for introducing new creatures into the moon's craters. By carefully considering SOLID principles, we have developed a solution that is maintainable and extensible, paving the way for future enhancements and extensions.

UML Diagram:



Design Goal:

The design goal for this requirement is to **simulate the dynamic behaviours of the creatures on the moon** while adhering closely to relevant design principles and best practices.

Design Decision:

In implementing the picking up scraps behaviour of the alien bug and the follow target behaviour, the decision was made to include all the logic needed in the `PickUpTrashBehaviour` and `FollowBehaviour` respectively. `PickUpTrashBehaviour` has the highest priority in the behaviour list, it will return pick up action if there is any item on the ground

When the alien bug has a target and there is no item on the ground it is currently standing on, it will always follow the target. This behaviour will check if there is any way to get closer to the target, if the target is already at its surroundings, it will do nothing.

The suspicious astronaut extends from `HostileCreature` with its own deadly intrinsic weapon.

Alternative Design:

One alternative approach could involve handling all the behaviour in the `allowableAction()` function. However, it will be too much responsibility for the `AlienBug` class to handle alone.

Analysis of Alternative Design:

The alternative design is not ideal because it violates various Design and SOLID principles:

1. Single Responsibility Principle:

- If all the logic is put inside `allowableAction`, the `AlienBug` will violate the SRP rules as it needs to take care of every behaviour that the bug has. The class will become unfocused and too populated.

2. Don't Repeat Yourself:

- Implementing the code inside the `AlienBug` instead of making them a separate class would prevent other creatures with similar behaviour in the future to use those behaviours. They will have to repeat the same code, causing repetition.

Final Design:

In our design, we closely adhere to SRP, DIP and DRY:

1. Single Responsibility Principle:

- **Application:** PickUpTrashBehaviour and FollowBehaviour will handle their own logic, this will remove Alien bug's responsibility of handling the pick up scraps and follow target implementation.
- **Why:** This will keep our classes focused and greatly increase our code maintainability.
- **Pros/Cons:** It promotes code clarity and maintainability.

2. Dependency Inversion Principle:

- **Application:** Both PickUpTrashBehaviour and FollowBehaviour implements the Behaviour interface, it creates a contract on how a behaviour should be, which is either returning an Action or not returning anything.
- **Why:** This allows the other part of the system to know that the behaviour will behave as agreed upon while not needing to worry about the specific code implementation, This will create code encapsulation which will enhance our code quality.
- **Pros/Cons:** It increases the code clarity but it might end up in too many classes as more and more behaviours are added.

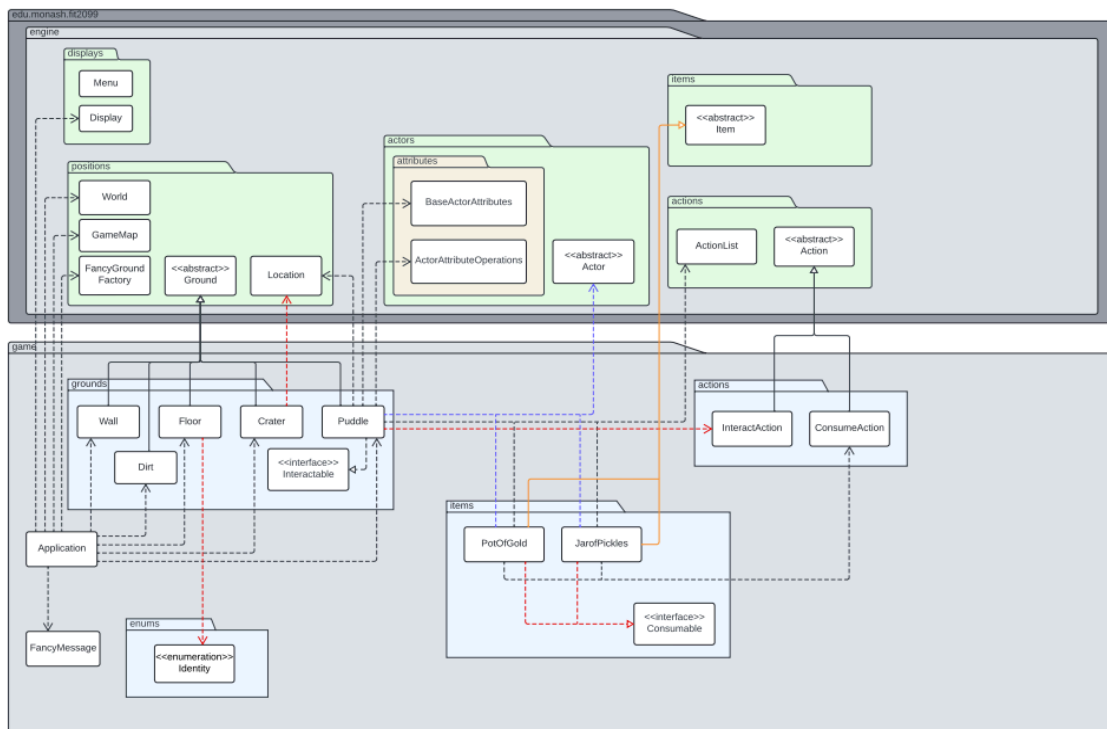
3. Don't Repeat Yourself:

- **Application:** If there is any other creature that has similar behaviour, they can easily add this behaviour to their behaviour lists.
- **Why:** Repetition of code will reduce the maintainability of the code, for example, if we made a logic error in the code with multiple repetitions, it will be hard to fix the bug for every one of them.
- **Pros/Cons:** It increases the code maintainability and clarity.

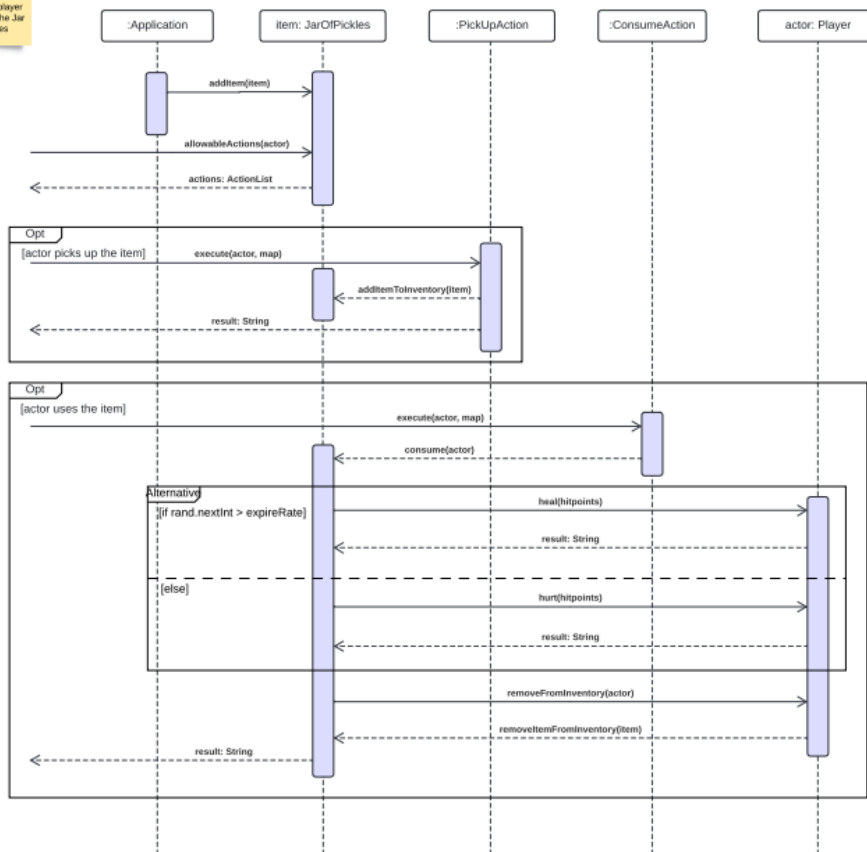
Conclusion:

Overall, our chosen design provides a robust framework for alien bug's and suspicious astronaut's behaviours. By carefully considering SRP, DIP and DRY, we have developed a solution that is efficient and maintainable, paving the way for future code maintenance and extensions.

UML Diagram:



This sequence diagram is specific to the scenario where the player consumes the Jar of Pickles



Design Goal:

The design goal for this requirement is to introduce additional scraps with their own usages and implement an action for drinking from puddles, while adhering closely to relevant design principles and best practices.

Design Decision:

In implementing the additional scraps, the decision was made to extend them from the Items abstract class and make use of the Consumables interface. Similar to some other scraps, they can be picked up and stored in the player's inventory (which can be used), thus abstraction is induced for better code sustainability.

The Puddle class would implement an interface Interactable, such that the player can interact with the puddle and drink water. This interface is created to allow efficient additions of Ground types that can also be interacted with. InteractAction will handle all actions that can be done by the player on the Interactables.

Alternative Design:

One alternative approach could involve enabling the puddles to generate an Item type Water at every tick, such that the player could interact with the puddle as an Item type instead. However, this approach will likely require constant processing power and memory allocation due to the abundance of generating water at every puddle. Implementing such that drinking from the puddle can be directly executed and results directly shown using an interface would minimise complexity and even allow better code structure for future maintenance.

Analysis of Alternative Design:

The alternative design is not ideal because it violates various Design and SOLID principles:

1. Open-Closed Principle:

- This design makes extending the behaviour of the puddle without modifying its existing code difficult. If the player would have additional or different interactions on the puddle, the features cannot be easily extended without the use of abstraction.

2. Dependency Inversion Principle:

- This approach heavily promotes tight coupling of the Puddle class with its concrete implementations which defines the behaviour of the puddle (generating water). Not depending on abstractions would reduce flexibility and code maintainability if there are additional features, or if there are other Ground types that have the same functionalities.

Final Design:

In our design, we closely adhere to SRP, OCP and LSP:

1. Single Responsibility Principle:

- **Application:** Creating an InteractAction class to handle actions that can be performed by the player on a particular Ground type.
- **Why:** Implementing the execution of actions in the Puddle class itself would defeat the purpose of classes having only one reason to change, thus violating the SRP. Having an additional class to handle the actions would ensure having only a single responsibility on each class.
- **Pros/Cons:** Doing so would provide much better understandability and code maintenance, however this would involve the creation of an additional class, thus excessive use may cause class proliferation

2. Open-Closed Principle:

- **Application:** Creating an interface Interactables to 'group up' all Ground types that can be interacted with by the player, while defining their potential functionalities.
- **Why:** This ensures that the classes involved are open for extension, without modifying any existing code. In this case, new functionalities of the Ground types can be added through new implementations of interfaces, such that previous features are untouched.
- **Pros/Cons:** Adhering to this principle can prevent blunders that may happen if we were to directly modify any existing code, thus avoiding bugs and errors. Although just like SRP, excessive use of interfaces may potentially result in interface proliferation.

3. Liskov Substitution Principle:

- **Application:** PotOfGold and JarOfPickles extends from the Item abstract class, with their methods overridable.
- **Why:** To reduce code redundancy, the objects of the Item class should be replaceable by its subclasses, thus introducing polymorphism. This allows some parts of the code to handle different types of objects, such that the subclasses can be used interchangeably.

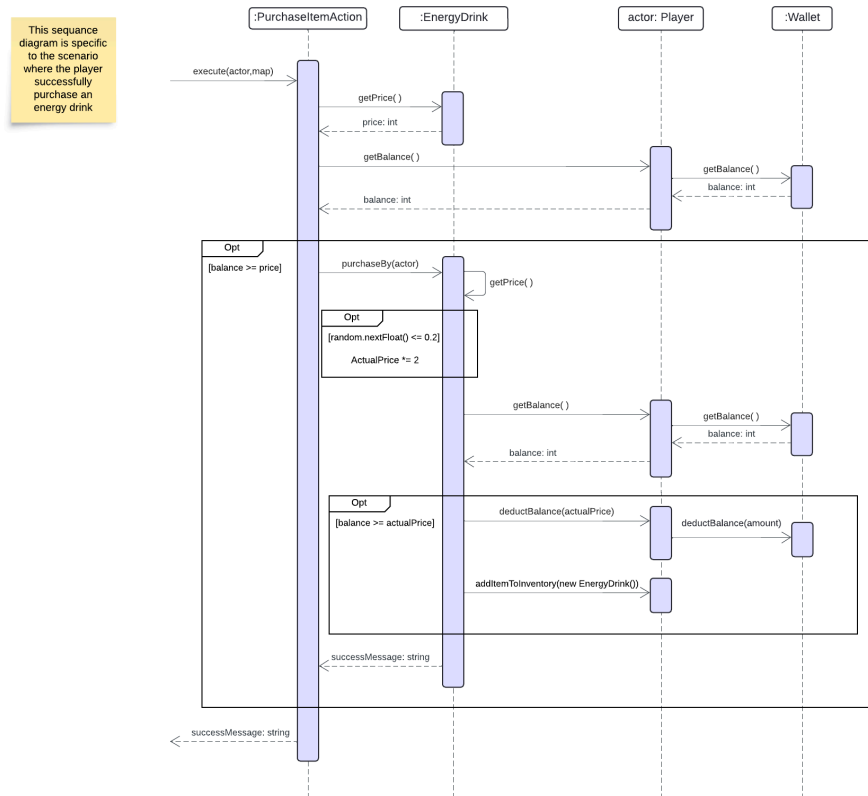
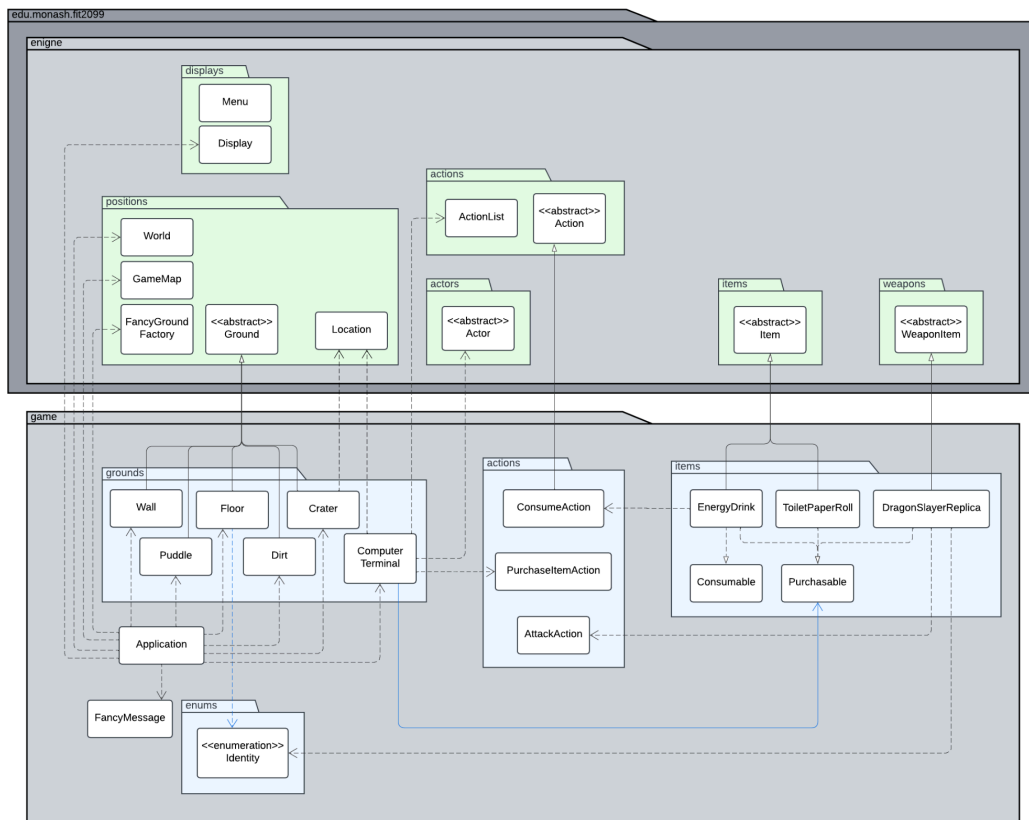
- **Pros/Cons:** This principle allows for easier extension of new subclasses and promotes the reusing of existing code. The downside is that adhering to LSP may introduce unwanted complexity and other developers may find it difficult to understand.

Conclusion:

Overall, our chosen design provides a robust framework for generating additional scraps and allowing consumption of water from puddles. By carefully considering the SOLID principles, we have developed a solution that is scalable and effective, paving the way for future extensions or modifications.

Requirement 4: Static factory's staff benefits

UML Diagram:



Design Goal:

The design goal for this assignment is to create a Computer Terminal for the intern to purchase items for exploration, while adhering closely to relevant design principles and best practices.

Design Decision:

In implementing the Computer Terminal for item purchases, the decision was made to extend the Ground abstract class, allowing the intern to interact with it and access a list of purchasable items. Each item is implemented as a separate class that implements the Purchasable interface, providing a consistent interface for handling purchases.

The PurchaseItemAction class encapsulates the logic for executing a purchase, checking the intern's balance, deducting the appropriate credits, and handling any random events or errors that may occur during the purchase process. This separation of concerns ensures that the purchase logic is decoupled from the specific items being purchased, promoting code reusability and maintainability.

Alternative Design:

One alternative approach could involve implementing a ComputerTerminal class responsible for both displaying available items and handling purchases. However, this approach would violate the Single Responsibility Principle (SRP) by combining multiple responsibilities within a single class. Additionally, it could lead to bloated code, especially as the system grows in complexity.

Analysis of Alternative Design:

The alternative design is not ideal because it violates various Design and SOLID principles:

1. Single Responsibility Principle:

- Combining the responsibilities of displaying available items and handling purchases in a single class violates the SRP, making the code harder to maintain.

Final Design:

In our design, we closely adhere to SRP, DIP, and ISP:

1. Single Responsibility Principle:

- **Application:** The ComputerTerminal is responsible for displaying available items, while the PurchaseItemAction class handles purchase logic.
- **Why:** PurchaseItemAction is a type of action that the player is allowed to perform on the purchasable. It is handled within the purchasables' class, so that the player class does not have to bother about all its items.
- **Pros/Cons:** This design choice simplifies debugging and future modifications but may require additional classes.

2. Dependency Inversion Principle:

- **Application:** The Purchasable interface decouples the PurchaseItemAction class from specific item implementations, enabling polymorphism.
- **Why:** The system depends on abstractions rather than concrete implementations, allowing for loose coupling and easier testing.
- **Pros/Cons:** This design choice enhances flexibility and scalability but requires careful consideration of abstraction and interface design.

3. Interface Segregation Principle:

- **Application:** The Purchasable interface only includes methods relevant to the purchase process, which are the getPrice and purchasedBy methods. It does not implement methods that it does not care about, such as consume and spawn.
- **Why:** Each interface is focused, preventing unnecessary dependencies.
- **Pros/Cons:** This design choice promotes code clarity and maintainability but may result in more interfaces or classes.

Conclusion:

Overall, our chosen design provides a robust Computer Terminal system for item purchases. This design ensures modularity, flexibility, and maintainability, laying a solid foundation for future enhancements and optimizations. This approach facilitates clean code organisation, promotes code reusability, and simplifies testing, ultimately enhancing the overall quality and scalability of the system.

Revised Code:

In assignment 1, the player class will be responsible to add actions to attack all the hostile creatures nearby using AttackBehaviour. However, behaviour can only return one Action.

In the revised code, the responsibility is removed from Player class and put into the HostileCreature class and rare individual classes that are hostile but with no attack ability, like Alien bug.